PUPC / CODESPRINT 1.0 / DAY 04

Greedy Algorithms

Saturday, 17 October 2026 · 10:00 AM–2:00 PM · Room 115
Presidency University

Ahmmad Nur Swapnil Lecturer, Department of CSE, Presidency University
B.Sc. in CSE, BUET · Codeforces Expert · ICPC Asia Dhaka Regional 2023 Participant
Codeforces: Ans2003 / Ans_Practice_2003

The greedy idea

A greedy algorithm repeatedly makes a **locally best feasible choice** and commits to it. It gives an optimal answer **only when the greedy choice can be justified**.

Example 1: When greedy works

Activity selection. Choose the maximum number of non-overlapping meetings. Meetings that touch at endpoints are compatible.

Greedy rule: Sort meetings by finishing time; take the next one that starts after (or exactly when) the last selected meeting ends.

Meeting	Time	Decision (in finish-time order)	Selected
A	[1,3)	Finish first	Yes
B	[2,5)	Overlaps A	No
C	[4,6)	Starts after A ends	Yes
D	[6,8)	Starts when C ends	Yes
E	[5,9)	Overlaps selected meetings	No
F	[8,10)	Starts when D ends	Yes

Result: A → C → D → F: **4 meetings**.

Why it works: An earliest-finishing available meeting leaves at least as much room for future meetings as any alternative first choice.

Example 2: When a greedy rule fails

Coin change. Coin denominations: $\{1, 3, 4\}$; target sum: 6. Try choosing the largest coin that fits each time.

Largest-first	$6 \to 4$ (2 left) $\to 1$ (1 left) $\to 1$	3 coins
Optimal	$3 + 3 = 6$	2 coins

Key point

A locally appealing choice is not always optimal. **State your greedy rule, look for counterexamples, and justify why your choice is safe.** For arbitrary coin denominations, dynamic programming may be needed.

Try a case where largest-first works: 996A — Hit the Lottery **800**
Coin denominations: 1, 5, 10, 20, 100. A quick optional warm-up, not part of the guided set.

CODESPRINT 1.0 / DAY 04

Classroom problem solving

8 guided problems (800–1100) + 1 special (1500) + 1 independent exercise (800). Click a title to open Codeforces.

A typical greedy workflow

Preprocess (sort/count) → **Choose and update** (greedy steps) → **Postprocess** (restore indices/output). Not every problem uses all three stages.

Example: in *Restore the Weather*, sort, match in sorted order, then restore the original positions.

Why is the choice safe? Exchange: swap without loss. Forced choice: alternatives cannot be optimal. Invariant: maintain a useful property.

01 / Feasibility and Resource Allocation

1. **Food for Animals** 1675A — **800**

02 / Sorting, Extremes, and Exchange Greedy

2. **Sasha and Array Coloring** 1843A — **800**
3. **Two Arrays And Swaps** 1353B — **800**

03 / Sorting, Matching, and Index Restoration

4. **Restore the Weather** 1833B — **900**

04 / Greedy Scans and Segment Boundaries

5. **Long Long** 1843B — **800**
6. **Luke is a Foodie** 1704B — **1000**

05 / Forced Greedy Decisions

7. **Make It Increasing** 1675B — **900**

06 / Maximum Marginal Gain First

8. **Line** 1722D — **1100**

SPECIAL CHALLENGE / Greedy Pairing and Two Pointers

9. **Price Maximization** 1690E — **1500**

SOLVE IT YOURSELF / Constructive Greedy

10. **Minimum Varied Number** 1714C — **800**

CODESPRINT 1.0 / DAY 04

Practice at home

18 additional Codeforces greedy problems (800–1200), grouped by technique.

01 / Feasibility and Priority Rules

GamingForces	1792A	**800**
Mark the Photographer	1705A	**800**
Array Decrements	1690B	**800**

02 / Sorting, Matching, and Construction

Taisia and Dice	1790B	**800**
Parity Sort	1851B	**800**
Promo	1697B	**900**
Mystic Permutation	1689B	**900**
Shoe Shuffling	1691B	**1000**
Eating Queries	1676E	**1100**

03 / Greedy Scans and Forced Operations

Restoring the Duration of Tasks	1690C	**800**
Mark the Dust Sweeper	1705B	**900**
Sort Zero	1712C	**1100**

04 / Greedy on Strings

Lex String	1689A	**800**
Deletive Editing	1666D	**900**
Comparison String	1837B	**900**

05 / Segments and Advanced Greedy

NIT Destroys the Universe	1696B	**900**
Array merging	1831B	**1000**
Virus	1704C	**1200**

Selection primarily from the instructor's accepted Codeforces submissions. Two classic problems (1353B and 996A) are added for teaching clarity. All included problems have the greedy tag on Codeforces.